

Portafolio 11.

<https://eliasvqz.github.io/mi-portafolio/>

Arquitectura Cloud Serverless Inteligente

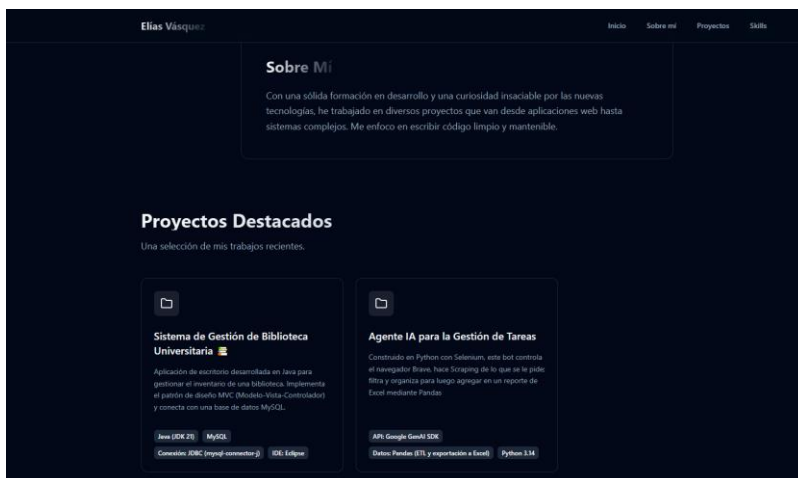
Aquí tienes la redacción profesional de tu tercer proyecto (el del Módulo 8 que me mostraste en el PDF). Lo dividí en dos partes:

1. El código exacto para que lo pegues en tu archivo `profile.json`.
2. La redacción larga (Caso de Estudio) por si quieres hacerle una página dedicada en tu portafolio.

1. Código para agregar a tu `profile.json`

```
{
  "name": "Arquitectura Cloud Serverless Inteligente ☁️",
  "description": "Diseño y despliegue de un backend 100% Serverless en AWS. Implementé endpoints HTTP con API Gateway que disparan funciones Lambda en Python, persistiendo datos en DynamoDB y alojando recursos estáticos en Amazon S3.",
  "url": "https://github.com/eliasvqz",
  "tags": [
    "AWS Lambda",
    "API Gateway",
    "Amazon DynamoDB",
    "Amazon S3",
    "Python (Boto3)"
  ]
}
```

Antes:



2. Redacción extendida (Caso de Estudio)

Usa estos textos si vas a crear un modal, una sección de detalles o si lo vas a documentar en el README de tu GitHub.

Breve descripción de la actividad:

Despliegue de una solución en la nube utilizando servicios administrados de AWS para crear una API funcional y un sitio web de alta disponibilidad, eliminando por completo la necesidad de gestionar servidores tradicionales.

Desafío principal:

El reto consistió en acoplar de forma segura distintos servicios nativos de la nube (almacenamiento, enrutamiento, cómputo y bases de datos NoSQL) asegurando que se comunicaran correctamente mediante roles y políticas de seguridad (IAM), logrando un entorno escalable automáticamente ante picos de tráfico.

Solución propuesta:

Se diseñó una arquitectura orientada a eventos. Un bucket de **Amazon S3** se configuró para el alojamiento estático. Las peticiones externas se gestionaron a través de **Amazon API Gateway**, definiendo rutas HTTP (/usuarios y /pedidos) que detonaban funciones **AWS Lambda** escritas en Python. La función de pedidos utilizaba la librería Boto3 para persistir la información directamente en una tabla de **Amazon DynamoDB**.

Herramientas técnicas utilizadas:

AWS Lambda, Amazon API Gateway, Amazon DynamoDB, Amazon S3, IAM, Python 3, Boto3.

Principales aprendizajes:

Comprendí el paradigma y los beneficios económicos del modelo Functions as a Service (FaaS) y pago por uso. Aprendí a gestionar permisos de ejecución específicos (Least Privilege) para que Lambda pudiera escribir en DynamoDB de forma segura.

Métricas de impacto logradas:

- **Reducción de costos de inactividad:** Al ser 100% Serverless, el costo de infraestructura cuando no hay tráfico es de \$0.
- **Escalabilidad instantánea:** La arquitectura es capaz de soportar desde 1 hasta miles de peticiones por segundo de manera nativa sin intervención manual.

Justificación de la elección:

Elegí este proyecto como caso de estudio principal porque demuestra mi capacidad para salir del código tradicional y diseñar infraestructuras Cloud modernas, rentables y alineadas con las prácticas actuales del mercado tecnológico.